

گرداننده هوشمند بازی‌های گروهی حضوری

مستند معماری و طراحی سیستم

اعضای تیم:

سید محمد طاها شاهرخ، عرشیا طهماسب نژاد، علی عزیزی

نسخه: ۱.۰.۰

تاریخ: خرداد ۱۴۰۵

فهرست مطالب :

3.....	معرفی و دید کلی
5.....	محدوده و دامنه (Scope)
5.....	داخل Scope (MVP - فاز ۱):
5.....	خارج از Scope (فازهای بعدی):
5.....	فازبندی:

معرفی و دید کلی

نام پروژه: گرداننده هوشمند بازی‌های گروهی حضوری (Smart Party Game Master)

خلاصه:

این پروژه یک پلتفرم دیجیتال برای مدیریت و اجرای بازی‌های گروهی حضوری است که به صورت محلی (بدون نیاز به اتصال دائمی به اینترنت) روی یک دستگاه مرکزی اجرا می‌شود. هدف اصلی، ساده‌سازی فرآیند برگزاری بازی‌ها، کاهش بار مدیریتی گرداننده، و ایجاد تجربه‌ای روان و حرفه‌ای برای دوستان و خانواده‌ها است.

مشکل:

برگزاری بازی‌های گروهی حضوری مانند مافیا، جاسوس، و پانتومیم نیازمند یک گرداننده است که باید:

- نقش‌ها را به صورت تصادفی و عادلانه تخصیص دهد
- زمان‌بندی را مدیریت کند
- قوانین را یادآوری کند
- حالت بازی را پیگیری کند

این فرآیند زمان‌بر، مستعد خطا، و گاهی موجب کاهش لذت بازی می‌شود.

راه‌حل:

یک سیستم وب‌محور (Django + DRF) که:

- گرداننده با یک حساب کاربری، بازی‌ها را مدیریت می‌کند
- بازیکنان بدون نیاز به ثبت‌نام یا نصب اپلیکیشن شرکت می‌کنند
- نقش‌ها به صورت خودکار و مخفیانه تخصیص داده می‌شود
- تایمر و وضعیت بازی به صورت خودکار مدیریت می‌شود
- نام دوستان برای بازی‌های آتی ذخیره می‌شود

اهداف پروژه

اهداف کسب و کار:

1. ساده سازی تجربه کاربری: کاهش ۸۰٪ زمان مدیریت دستی بازی توسط گرداننده
2. افزایش دسترسی: حذف موانع فنی برای بازیکنان (بدون نیاز به اینترنت یا نصب اپلیکیشن)
3. قابلیت مقیاس پذیری: طراحی معماری برای افزودن بازی های جدید با حداقل تغییر در کد اصلی
4. ایجاد پایگاه کاربری وفادار: ذخیره دوستان و تاریخچه بازی ها برای استفاده مکرر

اهداف فنی:

1. معماری تمیز و قابل نگهداری: استفاده از Service Layer Pattern و اصول SOLID
2. پایداری و امنیت: احراز هویت امن (JWT)، مدیریت Session، و جلوگیری از تقلب
3. تست پذیری: پوشش ۸۰٪+ تست های واحد و یکپارچگی
4. مستندسازی خودکار: استفاده از drf-spectacular برای OpenAPI Schema

شاخص های موفقیت (KPI):

- زمان راه اندازی یک بازی جدید > ۳۰ ثانیه
- نرخ خطا در تخصیص نقش = ۰٪
- زمان پاسخ دهی API > ۲۰۰ms
- رضایت کاربران (بازخورد کیفی) < ۵/۴.۵

محدوده و دامنه (Scope)

داخل Scope (MVP - فاز ۱):

- سیستم احراز هویت و مدیریت گرداننده
- مدیریت دوستان (افزودن، ویرایش، حذف، پیشنهاد)
- بازی «جاسوس» (Spy Game) شامل:
 - تخصیص خودکار نقش‌ها (جاسوس، مکان برای دیگران)
 - تایمر
 - مدیریت وضعیت بازی
 - تعیین برنده و حدس کلمه بعد از اتمام تایمر

خارج از Scope (فازهای بعدی):

- بازی مافیا
- بازی پانتومیم
- PWA برای تجربه موبایل
- چت صوتی/تصویری
- رتبه‌بندی و سیستم امتیازدهی
- حالت آنلاین (multiplayer غیرحضوری)

فازبندی:

خروجی	مدت زمان تخمینی	عنوان	فاز
قراردادهای کد، مخزن Git	۱ هفته	هماهنگی و تنظیمات تیم	۰
ERD ، User Stories ، Use Cases	۲ هفته	طراحی و تحلیل	۱
Django setup ، تست‌های اولیه CI/CD	۱ هفته	تنظیمات پروژه	۲
API بازی جاسوس +	۴ هفته	MVP پیاده‌سازی	۳
Test coverage ۸۰%+	۲ هفته	تست و رفع باگ	۴
نسخه ۱.۰ آماده استفاده	۲ هفته	UX و بهبودهای PWA	۵

تحليل ذينفعان

گرداننده (Host):

- نیازها: رابط ساده، کنترل کامل بازی، ذخیره دوستان
- دردها: نیاز به یادآوری قوانین، مدیریت زمان، انتخاب تصادفی عادلانه
- تعامل: وب اپلیکیشن (دسکتاپ/موبایل)

بازیکنان (Players):

- نیازها: دسترسی سریع به نقش، عدم نیاز به نصب اپلیکیشن
- دردها: نگرانی از عدم تصادفی بودن انتخاب نقش
- تعامل: نمایش نقش

ویژگی‌های کلیدی

ویژگی‌های اصلی:

1. بدون نیاز به اینترنت برای بازیکنان: تمام اطلاعات روی دستگاه مرکزی گرداننده
2. تخصیص خودکار و عادلانه نقش‌ها: الگوریتم تصادفی با امکان قفل کردن نقش خاص برای بازیکن خاص
3. تایمر هوشمند: زمان‌بندی خودکار مراحل بازی با قابلیت Pause/Resume
4. مدیریت دوستان: پیشنهاد سریع افراد قبلی، جلوگیری از تایپ مکرر

مزایای رقابتی:

- سادگی: بدون نیاز به آموزش، رابط کاربری شهودی
- انعطاف: افزودن بازی‌های جدید بدون تغییر معماری کلی
- امنیت: عدم نیاز به اشتراک‌گذاری اطلاعات شخصی بازیکنان
- عملکرد: اجرا بدون وابستگی به کیفیت اینترنت

User Scenarios

US-01 | ثبت نام و ورود

سناریو: گرداننده برای اولین بار وارد سیستم می شود.

1. Host به آدرس اپ می رود
2. روی «ثبت نام» کلیک می کند
3. نام کاربری، ایمیل و رمز عبور وارد می کند
4. سیستم حساب می سازد و توکن JWT برمی گرداند
5. Host وارد داشبورد می شود

سناریوی جایگزین – لاگین:

1. Host نام کاربری و رمز وارد می کند
2. سیستم توکن صادر می کند
3. وارد داشبورد می شود

خطا: اطلاعات اشتباه → پیام Invalid credentials

سناریو مشاهده و ویرایش پروفایل:

1. Host وارد بخش «پروفایل» می شود
2. سیستم اطلاعات فعلی نمایش می دهد: نام کاربری، ایمیل
3. Host روی «ویرایش» کلیک می کند
4. فیلد مورد نظر را تغییر می دهد
5. روی «ذخیره» کلیک می کند
6. سیستم اعتبارسنجی می کند و اطلاعات آپدیت می شود

US-02 | مدیریت دوستان

سناریو A – افزودن دوست:

1. Host وارد بخش «دوستان» می‌شود
2. نام دوست را تایپ می‌کند
3. سیستم نام را ذخیره می‌کند

سناریو B – ویرایش نام:

1. Host روی یک دوست کلیک می‌کند
2. نام را ویرایش می‌کند
3. سیستم آپدیت می‌کند

سناریو C – مشاهده لیست دوستان:

1. Host وارد بخش «دوستان» می‌شود
2. سیستم لیست تمام دوستان را نمایش می‌دهد
3. Host می‌تواند دوست مورد نظر را جستجو یا انتخاب کند

سناریو D – حذف دوست:

1. Host دوست را انتخاب و حذف می‌کند
2. سیستم از لیست حذف می‌کند (بازی‌های قبلی حفظ می‌شوند)

US-03 | ساخت بازی جدید (جاسوس)

سناریو:

1. Host روی «بازی جدید» کلیک می‌کند

2. بازی «جاسوس» را انتخاب می‌کند
 3. مدت تایمر را تنظیم می‌کند (مثلاً ۸ دقیقه)
 4. بازیکنان را اضافه می‌کند:
 - از لیست دوستان ذخیره شده انتخاب می‌کند، یا
 - نام جدید تایپ می‌کند
 5. تعداد جاسوس‌ها را تعیین می‌کند
 6. روی «شروع» کلیک می‌کند
- خطا – تعداد ناکافی بازیکن:** حداقل ۴ نفر لازم است → پیام خطا
- خطا – تعداد جاسوس بیش از حد:** نمی‌توان بیشتر از $\lfloor n/3 \rfloor$ جاسوس داشت

US-04 | نمایش نقش‌ها به بازیکنان

سناریو:

1. سیستم به صورت خودکار نقش‌ها را تخصیص می‌دهد
 2. گوشی دست به دست می‌چرخد
 3. هر بازیکن نامش را تأیید می‌کند
 4. نقشش نمایش داده می‌شود:
 - **جاسوس:** «تو جاسوسی! مکان را نمی‌دانی»
 - **بقیه:** «مکان: [نام مکان]»
 5. بازیکن صفحه را به بعدی می‌دهد
 6. بعد از آخرین نفر، تایمر شروع می‌شود
- خطا:** اگر بازیکنی بخواهد نقش دیگری را ببیند → سیستم اجازه نمی‌دهد

US-05 | مدیریت تایمر حین بازی

سناریو:

1. تایمر خودکار شروع می‌شود
2. Host می‌تواند:
 - **Pause**: زمان متوقف می‌شود
 - **Resume**: ادامه می‌یابد- **Stop** زودهنگام: بازی زودتر تمام می‌شود
3. وقتی تایمر به صفر می‌رسد → سیستم اعلام می‌کند «زمان تمام شد»

US-06 | پایان بازی و حدس جاسوس

سناریو A – اتمام تایمر:

1. تایمر تمام می‌شود
2. بازیکنان رأی می‌گیرند چه کسی جاسوس است
3. Host نتیجه رأی را وارد می‌کند
4. Host اعلام می‌کند حدس صحیح بود یا نه
5. سیستم برنده را ثبت می‌کند

سناریو B – جاسوس مکان را حدس می‌زند:

1. قبل از اتمام تایمر، جاسوس ادعای حدس مکان می‌کند
2. Host بازی را متوقف و حدس را ثبت می‌کند
3. سیستم صحت حدس را بررسی و نتیجه را اعلام می‌کند

سناریو C – بازیکنان جاسوس را درست شناسایی کردند:

جاسوس باید کلمه را حدس بزند

اگر درست حدس بزند:

جاسوس برنده است

اگر اشتباه حدس بزند:

بازیکنان برنده میشوند

سناریو D – جاسوس شناسایی نشد:

→ جاسوس برنده است

US-07 | مشاهده تاریخچه بازی‌ها

سناریو:

1. Host وارد بخش «تاریخچه» می‌شود
2. لیست بازی‌های قبلی نمایش داده می‌شود (تاریخ، نوع بازی، برنده)
3. Host روی یک بازی کلیک می‌کند و جزئیات می‌بیند

US-08 | لاگ‌اوت

1. Host روی «خروج» کلیک می‌کند
2. توکن invalidate می‌شود
3. به صفحه لاگین هدایت می‌شود

Use Case

UC-01 | Register & Login

فیلد	مقدار
Actor	Host
Precondition	—
Trigger	باز کردن اپ

Main Flow (Register):

1. POST /auth/register/ ← {username, email, password}
2. username و email اعتبارسنجی یکتایی
3. {access_token, refresh_token} برگرداندن → User ساخت

Alt Flow (Login):

1. POST /auth/login/ ← {username, password}
2. {access_token, refresh_token} اعتبارسنجی → برگرداندن

Alt Flow (Refresh Token):

1. POST /auth/token/refresh/ ← {refresh_token}
2. جدید access_token برگرداندن

Exception Flows:

- 401 → اشتباه Invalid credentials
- username/email 400 → تکراری Already exists
- 400 → فیلد خالی Validation error

Postcondition: Host است JWT احراز هویت شده با Host

UC-02 | Profile Management

فیلد	مقدار
Actor	Host
Precondition	احراز هویت شده

Main Flow (View):

1. GET /auth/profile/ → {username, email}

Alt Flow (Update):

1. PATCH /auth/profile/ ← {username?, email?}
2. اعتبارسنجی یکتایی
3. برگرداندن اطلاعات آپدیت شده

Alt Flow (Change Password):

1. POST /auth/change-password/ ← {old_password, new_password}
2. تأیید رمز قدیم → ذخیره رمز جدید

Exception Flows:

- username/email 400 → تکراری

- Wrong password رمز قدیم اشتباه → 400

UC-03 | Friend Management

فیلد	مقدار
Actor	Host
Precondition	احراز هویت شده

Main Flow (List & Search):

1. GET /friends/?search={query} → لیست دوستان

Alt Flow (Add):

1. POST /friends/ ← {name}
2. {id, name} ذخیره و برگرداندن

Alt Flow (Edit):

1. PATCH /friends/{id}/ ← {name}
2. آپدیت نام

Alt Flow (Delete):

1. DELETE /friends/{id}/
2. (soft reference) در بازی‌های قبلی حفظ می‌شوند Player رکوردهای - حذف از لیست

Exception Flows:

- نامعتبر → id404
- نام خالی → 400

UC-04 | Create Game Session (Spy)

فیلد	مقدار
Actor	Host
Precondition	احراز هویت شده
Invariant	$spies \leq \lfloor n/3 \rfloor$ ، $n \geq 4$

Main Flow:

1. POST /games/spy/sessions/ ←

content_copy json

```
{  
  "timer_duration": 480,  
  "spy_count": 1,  
  "players": [  
    {"friend_id": 5},  
    {"name": "علی"}  
  ]  
}
```

2. اعتبارسنجی تعداد بازیکنان و جاسوس ها

3. سیستم مکان تصادفی انتخاب می کند

4. (shuffle) نقش ها تخصیص می یابد

5. $session_id \rightarrow$ برگرداندن با status=ROLE_REVEAL GameSession ایجاد

Exception Flows:

- $n < 4 \rightarrow 400$ Minimum 4 players required
- $\text{spies} > \lfloor n/3 \rfloor \rightarrow 400$ Too many spies
- timer_duration خارج از بازه مجاز $\rightarrow 400$

UC-05 | Role Reveal

فیلد	مقدار
Actor	Host + Players
Precondition	<code>session.status == ROLE_REVEAL</code>

Main Flow

1. GET /games/spy/sessions/{id}/reveal/

→ لیست نام بازیکنانی که هنوز نقش نگرفت:

`pending_players{ ["علی", "سارا", "رضا"] :`

2. بازیکن روی اسم خودش میزند:

POST /games/spy /sessions/{id}/reveal/ ← { player_name: "علی" }

3. سیستم نقش را فقط به همان بازیکن نشان می‌دهد:

○ `role: "spy"` → {

○ `role: "civilian", location{ "کافه" }` → {

4. حذف می‌شود (سایرین این تغییر را در مرحله ۱ می‌بینند) `pending_players` نام بازیکن از

5. خالی شود `pending_players` تکرار تا

6. ، تایمر شروع می‌شود `status → IN_PROGRESS` بعد از آخرین نفر:

UC-06 | Timer Management

فیلد	مقدار
Actor	Host
Precondition	session.status == IN_PROGRESS

Main Flow (Auto):

- GET /games/spy /sessions/{id}/timer → /
remaining_seconds, status{

Alt Flow (Pause/Resume):

1. POST /games/spy /sessions/{id}/timer/pause/
2. POST /games/spy /sessions/{id}/timer/resume/

Alt Flow (Stop Early):

1. POST /games/spy /sessions/{id}/timer/stop/
2. status → VOTING

Auto Transition:

- remaining == 0 → status → VOTING

UC-07 | End Game

مقدار	فیلد
Host	Actor
session.status == VOTING	Precondition

Main Flow

مرحله ۱ - رأی گیری:

POST /games/spy /sessions/{id}/vote/ ← { "voted_player_id": 3 }

سیستم بررسی می کند آیا بازیکن رأی خورده جاسوس است:

- (جاسوس فرصت حدس دارد) status → SPY_GUESS جاسوس شناسایی شد
- , برنده: جاسوس status → FINISHED جاسوس شناسایی نشد

مرحله ۲ - حدس مکان (فقط اگر جاسوس شناسایی شد):

POST /games/spy /sessions/{id}/spy-guess/ ← { "location_guess": "کافه" }

- , برنده: جاسوس status → FINISHED حدس درست
- , civilians, برنده: status → FINISHED حدس غلط

Result Logic(کامل)

برنده	حالت
جاسوس 🕵️	جاسوس شناسایی نشد

برنده	حالت
جاسوس 🕵️	جاسوس شناسایی شد + حدس درست
Civilians 🧑🧑	جاسوس شناسایی شد + حدس غلط
Civilians 🧑🧑	جاسوس شناسایی شد + حدس نزد (timeout)

Exception Flows

شرط

پاسخ

spy-guess در وضعیت غیر از SPY_GUESS 409 Conflict

تا اوت حدس جاسوس

auto → FINISHED, برنده: civilians

UC-08 | Game History

مقدار	فیلد
Host	Actor
احراز هویت شده	Precondition

Main Flow:

1. GET /games/spy /sessions/?status=FINISHED →

content_copy json

[{"id", "game_type", "played_at", "winner", "player_count"}]

Alt Flow (Detail):

1. GET /games/spy /sessions/{id} → جزئیات کامل (بازیکنان، نقش‌ها، مکان، نتیجه)

UC-09 | Logout

مقدار	فیلد
Host	Actor
احراز هویت شده	Precondition

Main Flow:

1. POST /auth/logout/ ← {refresh_token}
2. می‌کند blacklist refresh token را سیستم
3. 200 OK

Exception Flows:

- توکن نامعتبر/منقضی → 400

Session Status State Machine

CREATED → ROLE_REVEAL → IN_PROGRESS → VOTING → FINISHED

↘ (stop early) ↗

ERD



